Controle do PC pelo celular

Guia de aula: tudo o que foi construído, por que foi feito assim e o que cada parte significa.

Preparado para Willian – 23 de outubro de 2026

O que você vai encontrar aqui

- **1.** Visão geral e mapa dos arquivos
- **2.** Conceitos de base: cliente, servidor, IP, porta, HTTP, JSON
- **3.** O servidor em Python (Flask), recurso por recurso
- **4.** Segurança: o que protege seu PC e quais são os limites
- **5.** A interface: HTML, CSS e JavaScript
- **6.** Rodar escondido e ligar junto com o Windows
- **7.** Ligar o PC: Wake-on-LAN
- **8.** O app Android: WebView, Gradle e APK
- **9.** Problemas que apareceram e como foram resolvidos
- **10.** Limitações, exercícios para praticar e glossário

Como usar este guia

Cada capítulo explica a ideia primeiro e mostra um trecho real do seu código depois. Você não precisa decorar nada: a meta é que, ao abrir o `app.py` ou o `index.html`, você reconheça o que cada pedaço faz.

1. Visão geral

Você queria controlar o PC pelo celular (desligar, teclado, mouse, volume, abrir programas) e nunca achou nada pronto que funcionasse no seu computador. Então construímos do zero. O projeto tem três peças que conversam entre si:

Peça	O que é	Onde roda
Servidor	Um programa em Python (Flask) que fica esperando pedidos e executa as ações no Windows.	No PC
Interface	A página com os botões (HTML, CSS e JavaScript). O próprio servidor entrega essa página.	No celular, dentro do navegador ou do app
App Android	Uma casca (WebView) que abre a interface em tela cheia, como se fosse um app de verdade.	No celular

O caminho de um toque no celular até o PC é sempre o mesmo:

```
 CELULAR                          PC (Windows)
+-------------------+             +-------------------------+
| voce toca o botao |  pedido POST | Servidor Flask (:5000)  |
| "Desligar"        | -----------> | confere o token e       |
|                   |              | roda o comando shutdown |
| recebe a resposta | <----------- | responde em JSON        |
+-------------------+             +-------------------------+
```

Mapa dos arquivos

Arquivo	Para que serve
`app.py`	O servidor: todas as rotas, a segurança e as ações no PC.
`index.html`	A interface inteira (estrutura, visual e lógica dos botões).
`atalhos.json`	Lista dos seus atalhos (YouTube, Spotify, Discord...). Você edita sem mexer no código.
`requirements.txt`	Lista das bibliotecas Python que o projeto usa (flask, pynput, psutil...).
`token.txt`	Sua senha de acesso, criada na primeira vez que o servidor liga.
`iniciar.bat`	Instala as bibliotecas e liga o servidor com a janela preta visível (bom para ver erros).
`iniciar_oculto.vbs`	Liga o servidor sem janela.
`instalar_inicio.bat` / `remover_inicio.bat`	Colocam / tiram o servidor da inicialização do Windows.
`parar.bat`	Desliga o servidor que está rodando escondido.
`ligar.py`	Manda o sinal de Wake-on-LAN para ligar um PC desligado.
`manifest.json`, `icone.png`	Nome e ícone do app quando você adiciona o site à tela inicial.
`ControlePC-Android/`	O projeto do Android Studio que gerou o APK.

2. Conceitos de base

Cliente e servidor

Pense num restaurante. O cliente faz o pedido, o servidor (a cozinha) prepara e devolve. No seu projeto, o **celular é o cliente** e o **PC é o servidor**. O PC fica parado esperando; quando chega um pedido, ele faz o que foi pedido e responde se deu certo.

IP e porta

O **IP** é o endereço do aparelho na rede (como o endereço de uma casa): no seu caso, `192.168.0.100`. A **porta** é a porta de entrada de cada programa naquele aparelho (como o número do apartamento). Juntos formam o endereço completo: `http://192.168.0.100:5000`.

- `127.0.0.1` significa “neste mesmo computador”. O servidor acessado só por ele não seria visto pelo celular.
- `0.0.0.0` significa “aceite conexões de qualquer aparelho da minha rede”. É o que colocamos no `app.run`, e por isso o celular consegue entrar.

Rede local e internet

Endereços que começam com `192.168` são **privados**: existem só dentro da sua casa. É por isso que o gerador de APK online (Appy Pie) não serviria: os servidores dele estão na internet e não conseguem enxergar um endereço que só existe na sua rede.

HTTP, rotas, GET e POST

O **HTTP** é a língua que navegador e servidor falam. Cada funcionalidade vira uma **rota**, que é um endereço dentro do servidor, como `/api/volume` ou `/api/tela`. Os dois tipos de pedido que importam aqui:

- **GET**: “me mostre algo”. É o que o navegador faz quando você digita um endereço.
- **POST**: “execute esta ação”. Usamos POST em todos os comandos. Assim, ninguém desliga seu PC só por clicar num link ou abrir um endereço por engano.

JSON e API

JSON é um formato de texto para trocar informação, por exemplo `{"tecla": "enter"}`. O conjunto de rotas que o servidor oferece se chama **API**. Repare numa consequência disso: a interface é só mais um cliente da API. Um script Python seu, ou outro app, poderia chamar as mesmas rotas.

3. O servidor em Python (app.py)

3.1 Flask e rotas

O **Flask** é uma biblioteca que transforma funções Python em rotas web. A linha que começa com `@app.post` é um *decorador*: ela liga o endereço à função logo abaixo. O `jsonify` devolve a resposta em JSON.

```
app = Flask(__name__)

@app.post("/api/ping")
def ping():
    return jsonify(msg="Conectado")
```

Rota	O que faz	Biblioteca
/api/desligar, reiniciar, bloquear, suspender, cancelar	Energia do PC	subprocess
/api/desligar_em	Desliga depois de X minutos	subprocess
/api/tecla, /api/texto	Teclas especiais e digitar texto	pynput
/api/mouse	Mover, clicar e rolar	pynput
/api/volume	Ler e definir o volume	pycaw
/api/status	Uso de processador, memória e disco	psutil
/api/tela	Captura da tela como imagem	Pillow (ImageGrab)
/api/atalhos, /api/abrir	Lista e abre seus atalhos	os, pathlib
/api/area	Copia (e cola) texto no PC	pyperclip + pynput

3.2 Energia: desligar, reiniciar, bloquear

O Windows já tem um programa chamado `shutdown`. O servidor só precisa "digitar" esse comando por você, usando `subprocess`. Os parâmetros: `/s` desliga, `/r` reinicia, `/t 10` espera 10 segundos e `/a` aborta (cancela).

```
ACOES = {
    "desligar": (["shutdown", "/s", "/t", "10"], "Desligando em 10 segundos"),
    "cancelar": (["shutdown", "/a"], "Desligamento cancelado"),
    "bloquear": (["rundll32.exe", "user32.dll,LockWorkStation"], "PC bloqueado"),
}

@app.post("/api/<acao>")
def energia(acao):
    comando, mensagem = ACOES[acao]
    subprocess.Popen(comando)
    return jsonify(msg=mensagem)
```

- Os **10 segundos** de espera são uma rede de segurança: se você tocou sem querer, dá tempo de cancelar.
- `Popen` dispara o comando e não espera ele terminar, então o servidor não fica travado.

- O <acao> na rota é uma **variável na URL**: um único pedaço de código atende várias rotas. Como o dicionário só tem ações conhecidas, qualquer outra coisa recebe erro 404.
- **Suspender** usa a função `SetSuspendState`. Se a hibernação estiver ligada no Windows, ele pode hibernar em vez de suspender.

3.3 Teclado e mouse

A biblioteca **pynput** simula o teclado e o mouse, como se você estivesse sentado no PC. Para o mouse, o celular manda apenas quanto o dedo andou (dx, dy) e o PC move o cursor *a partir de onde ele está*: é o movimento relativo, igual ao de um touchpad.

```
TECLAS = {"enter": Key.enter, "volume_mais": Key.media_volume_up, ...}

teclado.tap(Key.enter)        # aperta e solta
teclado.type("ola")           # digita um texto
mouse_ctl.move(dx, dy)        # move relativo
mouse_ctl.scroll(0, n)        # rola a página
```

> **Ideia importante: lista de permitidos**
>
> O celular nunca manda “o que executar”, só um **nome** (“enter”). O servidor procura esse nome no dicionário TECLAS e recusa o que não conhecer. Essa regra é uma das mais importantes em segurança: nunca execute diretamente o que vem de fora; valide contra uma lista.

3.4 Volume e status do PC

O volume do Windows é controlado pela biblioteca **pycaw**, que conversa com o sistema de áudio por uma tecnologia antiga do Windows chamada COM. Por isso o código chama `CoInitialize` a cada pedido: cada “fio” (thread) do servidor precisa avisar o Windows que vai usar o COM. O volume é um número de 0.0 a 1.0, e a interface converte de e para porcentagem.

O status usa a **psutil**. O `cpu_percent(interval=0.3)` mede o processador durante 0,3 segundo e devolve a média nesse intervalo; memória e disco são leituras instantâneas.

3.5 Tela do PC

O servidor tira uma foto da tela (`ImageGrab.grab()`), reduz para no máximo 1280x720 e salva como JPEG de qualidade 55. Imagem menor significa menos dados no Wi-Fi e atualização mais rápida. No modo automático, a interface repete o pedido a cada 3 segundos. Limitação: só captura o monitor principal e não mostra nada útil com o PC bloqueado.

3.6 Atalhos e abrir programas

Os atalhos ficam no `atalhos.json`. Cada item tem um **nome** (o texto do botão) e um **alvo** (o que abrir).

```
[
  {"nome": "YouTube", "alvo": "https://www.youtube.com"},
  {"nome": "Spotify", "alvo": "spotify:"},
  {"nome": "Discord", "alvo": "app:Discord"}
]
```

- Links e protocolos (`https://`, `spotify:`) abrem direto com `os.startfile`.

- O prefixo `app:` faz o servidor procurar o programa pelo nome no Menu Iniciar. Ele varre as pastas do Menu Iniciar atrás de arquivos `.lnk` (os atalhos) com `rglob`, prefere o nome exato e abre o atalho achado.
- Primeiro tentamos listar os apps pelo PowerShell (`Get-StartApps`), mas não funcionou com programas instalados por download. Varrer o Menu Iniciar funcionou, e o método antigo ficou só como plano B.
- **Segurança:** o celular manda só o *número* do atalho, e o servidor lê o caminho do arquivo. Ou seja, o celular não consegue mandar abrir qualquer coisa, só o que você listou.

3.7 Texto para o PC

O campo “Enviar para o PC” usa a **pyperclip** para copiar o texto na área de transferência do PC. O botão “Copiar e colar” também simula o Ctrl+V, com `with teclado.pressed(Key.ctrl): teclado.tap("v")`, ou seja, segura o Ctrl, aperta o V e solta.

4. Segurança

Este servidor dá controle sobre o seu PC. Por isso a segurança não foi um detalhe no final: ela está em camadas, cada uma cobrindo uma falha possível das outras.

Camada	Como funciona	Protege de
Token	Toda ação precisa do cabeçalho `X-Token` igual ao do `token.txt`.	Qualquer pessoa na sua rede que descubra o endereço
Só rede local	O servidor recusa (erro 403) conexões de IPs que não sejam privados ou do Tailscale.	Acesso vindo da internet por engano
Limite de tentativas	5 tokens errados bloqueiam aquele aparelho por 1 minuto.	Quem tenta adivinhar o token
Só POST	As ações não respondem a GET.	Desligar o PC ao abrir um link
Lista de permitidos	Teclas e atalhos vêm de listas fechadas no servidor.	Executar comandos escolhidos por quem não devia
Token fora do Git	O `.gitignore` impede o `token.txt` de subir para o GitHub.	Vazar a senha publicando o projeto

```
@app.before_request
def proteger():
    ip = ipaddress.ip_address(request.remote_addr)
    if not (ip.is_private or ip.is_loopback or ip in REDE_TAILSCALE):
        abort(403)
    ...
    if not hmac.compare_digest(enviado, TOKEN.encode()):
        falhas[request.remote_addr] = (erros + 1, agora + 60)
        return jsonify(msg="Token incorreto"), 401
```

O `before_request` roda **antes de qualquer rota**, então é um único guarda para o servidor inteiro, e não dá para esquecer de proteger uma rota nova. O `compare_digest` compara textos em tempo constante, o que impede que alguém descubra o token medindo o tempo que o servidor leva para responder. O token nasce de `secrets.token_urlsafe`, que usa aleatoriedade própria para segurança.

Os limites, sem enfeite

1) O site usa HTTP, que não é criptografado: na sua rede de casa o risco é baixo, mas em Wi-Fi público o token poderia ser lido por outra pessoa. Não use o app em redes que você não controla. **2)** Quem tiver o token e estiver no seu Wi-Fi controla o PC, então trate-o como senha. **3) Nunca abra a porta 5000 no roteador.** Para usar fora de casa, use o Tailscale (uma VPN, que criptografa tudo).

5. A interface (index.html)

A página junta três linguagens, cada uma com um papel:

Linguagem	Papel	Exemplo no projeto
HTML	Estrutura: o que existe na página.	Os botões, o campo de texto, as abas
CSS	Visual: cores, tamanhos, espaços.	O tema vermelho e preto, os cartões arredondados
JavaScript	Comportamento: o que acontece ao tocar.	Enviar o pedido ao servidor e mostrar o aviso

Falando com o servidor

Todo toque termina numa chamada `fetch`, que envia o POST com o token. O `async/await` faz o código esperar a resposta sem travar a tela.

```
const r = await fetch('/api/' + rota, {
  method: 'POST',
  headers: {'Content-Type': 'application/json', 'X-Token': token},
  body: JSON.stringify(corpo)
});
```

O token fica salvo no `localStorage` do navegador (por isso você digita só uma vez). Se o servidor responde 401, a página apaga o token salvo e pede de novo.

Truques de interface que valem aprender

- **Variáveis de CSS** (`:root`): as cores ficam definidas num só lugar. Trocar de verde-azulado para vermelho foi mudar uma linha, e não o arquivo todo.
- **Toque duplo para ações perigosas:** Desligar e Reiniciar mudam para “Toque de novo” por 3 segundos antes de enviar.
- **Touchpad com pointer events:** a página mede quanto o dedo andou e multiplica por 1,8 para o cursor andar mais. Um toque rápido (menos de 6 pixels e 250 ms) vira clique.
- **Throttle (acelerador):** o mouse manda no máximo um pedido a cada 40 ms. Sem isso, cada pixel do movimento viraria um pedido e o Wi-Fi engasgaria. O resto da divisão fica acumulado para o movimento não perder precisão.
- **Debounce (esperar parar):** o controle de volume só envia 120 ms depois que você para de mexer, e não a cada valor por onde o dedo passa.
- **Polling com economia:** o status se atualiza a cada 5 segundos, mas só se a página estiver visível (`document.hidden`).

- **Abas** embaixo: três grupos de recursos em vez de uma página enorme.
- **manifest.json + ícone:** permitem “Adicionar à tela inicial” com nome e ícone próprios.

6. Rodar escondido e com o Windows

No começo o servidor abria numa janela preta, e fechar essa janela desligava tudo. Resolvemos em três passos:

Arquivo	O que faz	Como
`iniciar_oculto.vbs`	Liga o servidor sem janela.	Um script VBS roda `cmd /c python app.py` com o modo de janela 0, que significa invisível.
`instalar_inicio.bat`	Liga o servidor sempre que o Windows inicia.	Usa o PowerShell para criar um atalho do .vbs na pasta Inicializar do seu usuário.
`parar.bat`	Desliga o servidor escondido.	Acha o processo que está escutando a porta 5000 e encerra só ele.
`remover_inicio.bat`	Desfaz a inicialização automática.	Apaga o atalho da pasta Inicializar.

Por que o parar.bat procura pela porta?

Poderíamos usar `taskkill /im python.exe`, mas isso encerraria **todos** os programas Python abertos, inclusive os seus estudos. Procurar quem escuta a porta 5000 derruba apenas o servidor do controle.

Detalhe do Windows que confundiu: ele esconde a extensão dos arquivos. Por isso o `iniciar.bat` aparecia só como “iniciar”. A coluna **Tipo** do Explorador mostra o que cada arquivo realmente é.

7. Ligar o PC: Wake-on-LAN

Aqui está uma pegadinha de lógica: o servidor é um programa, e um PC desligado não executa programa nenhum. Então o site **não tem como** ligar o próprio PC. A solução é o **Wake-on-LAN**: a placa de rede continua com um pouco de energia e fica ouvindo um pacote especial, o *magic packet*. Quando ele chega, o PC liga.

```
mac = sys.argv[1].replace(":", "").replace("-", "")
pacote = bytes.fromhex("FF" * 6 + mac * 16)      # 6 bytes FF + MAC repetido 16x
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.setsockopt(socket.SOL_SOCKET, socket.SO_BROADCAST, 1)
    s.sendto(pacote, ("255.255.255.255", 9))      # UDP, para todos da rede
```

- O pacote é enviado por **UDP broadcast** (para todos os aparelhos da rede), e só a placa com aquele MAC reage.
- Precisa ativar o Wake-on-LAN na BIOS e na placa de rede (Gerenciador de Dispositivos).
- Quem envia precisa estar **sempre ligado** na mesma rede (um Raspberry Pi, outro PC) ou ser um app de celular que mande o pacote. Esse trecho não foi testado no seu PC.

8. O app Android

A ideia: WebView

Um **WebView** é um navegador embutido dentro de um app. O nosso app tem uma tela só, que é um WebView em tela cheia abrindo `http://192.168.0.100:5000`. Como a interface é a própria página do servidor, o app mostra exatamente o que o navegador mostrava, e **qualquer mudança no site aparece no app sem gerar APK de novo**. O APK só precisa ser refeito se você mudar a casca (ícone, nome, comportamento).

Os arquivos do projeto Android

Arquivo	Função
`AndroidManifest.xml`	A “carteira de identidade” do app: nome, ícone, permissões e qual tela abre primeiro. Pede a permissão INTERNET e libera `usesCleartextTraffic`.
`MainActivity.java`	A tela do app: cria o WebView, liga JavaScript e armazenamento (para guardar o token), guarda o endereço e mostra o aviso quando não conecta.
`styles.xml`	Cores da barra de status e fundo (preto).
`mipmap-*/ic_launcher.png`	O ícone em cinco tamanhos, para telas de densidades diferentes.
`build.gradle` (3 arquivos)	As instruções de montagem: versões, nome do pacote e quais ferramentas usar.

- **Cleartext:** desde o Android 9, apps não aceitam HTTP sem criptografia por padrão. Liberamos porque o servidor está na sua rede local e não tem certificado HTTPS.
- **SharedPreferences:** uma pequena “gaveta” do app onde o endereço do PC fica salvo. Se o IP mudar, o app avisa e deixa você trocar sem recompilar.
- **setDomStorageEnabled(true):** sem isso o `localStorage` da página não funciona e você teria que digitar o token toda vez.

O caminho do código até o APK

O Android Studio não é mágico: ele organiza ferramentas que juntas transformam o código no arquivo instalável.

Termo	O que é
Android SDK	O kit com as ferramentas e as versões do Android necessárias para compilar. Foi o que o assistente de instalação baixou.
Gradle	O sistema de montagem: lê os `build.gradle`, baixa o que falta, compila e empacota. A “sincronização” demorada foi ele trabalhando.
APK	O pacote instalável do app (como o .exe do Windows). O nosso tem 34 KB porque é só a casca, sem o site dentro.
APK debug	Versão de teste, assinada com uma chave automática. Serve para instalar no seu celular. Para publicar na Play Store seria preciso uma versão release com chave própria.

Termo	O que é
Emulador	Um celular simulado no PC. Desmarcamos porque você testa no aparelho de verdade, o que economizou uns 2 GB.
Fontes desconhecidas / Play Protect	O Android avisa quando instalamos um APK que não veio da loja. Como você mesmo gerou o app, é seguro seguir.

9. Problemas que apareceram

Errar faz parte de programar, e entender o porquê de cada erro ensina mais do que o código pronto. Os que apareceram:

Problema	Causa	Solução
Não achava o `iniciar.bat`	O Windows esconde a extensão dos arquivos.	Olhar a coluna Tipo (“Arquivo em Lotes do Windows”).
Avisos amarelos no pip	Os scripts foram instalados numa pasta fora do PATH.	Ignorar, não afeta o servidor.
Celular não abria o site	O Firewall do Windows bloqueia por padrão.	Permitir o Python em redes privadas.
Apps não abriam pelos atalhos	O `Get-StartApps` não lista programas instalados por download.	Procurar o atalho `.lnk` no Menu Iniciar.
Gerador de APK online	Pede um endereço público, e o seu é de rede local.	Criar o app no Android Studio.
Android Studio não sincronizava	Foi aberta a pasta de fora do zip (havia outra pasta dentro).	Abrir a pasta que tem o `settings.gradle`.
Aviso do Gradle com Java 25	O Gradle 8.7 do projeto não suporta Java acima de 21.	Escolher o JVM 21, que vem com o Android Studio.
Token lido de um print	Letras parecidas (l, I, 1) podem se confundir.	Copiar do `token.txt`.

10. Limitações e próximos passos

O que ficou de fora ou não foi testado

- **LEDs:** a controladora usa controle remoto (infravermelho), e o PC não participa disso. Controlar pelo PC exigiria trocar a controladora por uma com Wi-Fi ou USB, ou montar um ESP32 imitando o controle. Deixamos de fora por decisão sua.
- **Wake-on-LAN e Tailscale** foram preparados, mas não testados no seu PC.
- **IP que muda:** se o roteador trocar o IP do PC, o app para de conectar. Fixar o IP do PC no painel do roteador resolve.
- **Servidor de desenvolvimento:** o Flask avisa na janela que não é para produção. Para uso pessoal em casa é suficiente.

- **HTTP sem criptografia:** veja o capítulo 4.

Exercícios para praticar

- **Fácil:** adicione um atalho novo no `atalhos.json` (outro programa ou site) e veja o botão aparecer.
- **Fácil:** troque a cor de destaque mudando `--accent` no `index.html`.
- **Médio:** crie uma ação nova no dicionário `ACOES` que abra o Gerenciador de Tarefas (`["taskmgr"]`) e um botão com `data-rota` para ela.
- **Médio:** crie uma rota `/api/hora` que devolva a hora do PC e mostre no aviso da tela.
- **Desafio:** mude a porta para 5050 e descubra tudo que precisa mudar para o app continuar funcionando.
- **Desafio:** pesquise como usar HTTPS com o Tailscale e como isso resolveria o limite do HTTP.

Glossário rápido

Termo	Significado
API / rota	O conjunto de endereços que o servidor oferece, e cada um deles.
Token	Uma senha longa e aleatória que prova que o pedido é seu.
POST / GET	Tipos de pedido HTTP: ação / leitura.
JSON	Formato de texto para trocar dados entre programas.
LAN	Rede local (a da sua casa).
Firewall	Filtro que decide quais conexões podem entrar no PC.
subprocess	Biblioteca do Python para rodar comandos do sistema.
Throttle / debounce	Limitar a frequência de um evento / esperar o evento parar antes de agir.
WebView	Navegador embutido dentro de um app.
PWA / manifest	Site que pode ser adicionado à tela inicial com nome e ícone próprios.
Gradle / SDK / APK	Sistema de montagem / kit de ferramentas do Android / arquivo instalável.

Para fechar

O que você tem hoje é um sistema completo: servidor, interface, segurança em camadas e um app instalável. Cada pedaço usa conceitos que aparecem em qualquer projeto web e mobile, então o que você aprendeu aqui serve bem além deste projeto.